

MONITOR Z · L04

ZTKMONS v0 en DESA: ensamblar y la demo en modo spool

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L04 — ZTKMONS v0 en DESA: ensamblar y la demo en modo spool

Objetivo: convertir el fuente de ZTKMONS en un programa que corre, leer sus errores, correr la demo con los jobs de prueba y **contestar con datos** las cinco preguntas abiertas. Es la **primera validación real** del monitor.

1. De fuente a programa

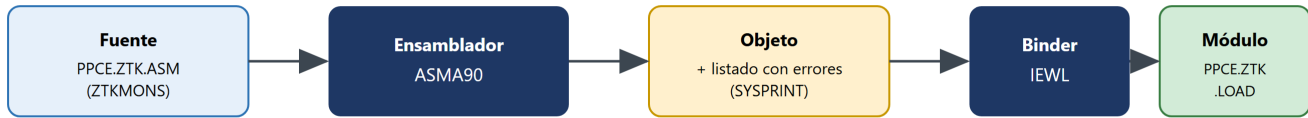
Paso	Programa	Entra	Sale
Ensamblar	ASMA90 (HLASM)	PPCE.ZTK.ASM(ZTKMONS)	Módulo objeto + listado con errores (SYSPRINT)
Enlazar	IEWL (binder)	El objeto	Módulo de carga PPCE.ZTK.LOAD(ZTKMONS)

Así lo hace `vivo\host\ZTKMONA.jcl`, o tu procedimiento habitual de ZTK si preferís:

```
//ASM      EXEC PGM=ASMA90,PARM='OBJECT,NODECK,XREF(SHORT)'  
//SYSLIB   DD DSN=SYS1.MACLIB,DISP=SHR          macros (STIMER, OPEN, I  
//        DD DSN=SYS1.MODGEN,DISP=SHR          mapeos de sistema (CVT,  
//SYSLIN   DD DSN=&&OBJ,DISP=(NEW,PASS),...      el objeto, temporal  
//SYSIN    DD DSN=PPCE.ZTK.ASM(ZTKMONS),DISP=SHR  
//LKED     EXEC PGM=IEWL,COND=(4,LT,ASM),        no enlaza si ASMA90 dio  
//        PARM='LIST,XREF,AMODE=31,RMODE=24'  
//SYSLMOD  DD DSN=PPCE.ZTK.LOAD(ZTKMONS),DISP=SHR
```

- **SYSLIB con MACLIB y MODGEN:** si falta una de las dos, el ensamblador no encuentra una macro o un mapeo, y los errores se encadenan.
- **COND=(4,LT,ASM) :** el binder no corre si el ensamblado dio más de 4. Así no se reemplaza un módulo bueno por uno roto.
- **AMODE=31,RMODE=24** en el binder: lo explicamos en L01.

 Fuente: `vivo\host\ZTKMONA.jcl` · *HLASM Programmer's Guide · z/OS MVS Program Management: User's Guide and Reference* (binder).



El job de ensamblado es vivo\host\ZTKMONA.jc1 (o tu procedimiento habitual de ZTK). Se corre con ZTKMONR.jc1.

RC de ASMA90	Qué significa	Qué hacés
0	Sin mensajes	Seguir
4	Advertencias	Leerlas; en general se sigue
8	Errores: probablemente el programa no hace lo que pensás	Corregir
12 / 16	Errores graves / terminales	Corregir

Analogía: la receta, la cocina y el plato

La fuente es la **receta**. El ensamblador la **traduce** a ingredientes preparados (el objeto) y te devuelve una lista de lo que no entendió. El binder **arma el plato** listo para servir (el módulo de carga), que es lo que el job ejecuta.

Fuente: HLASM Programmer Guide (códigos de retorno del ensamblador) · vivo\host\LEEME.md

Lámina L04-01_de_fuente_a_programa

2. Leer el listado de ASMA90

RC de ASMA90	Qué significa	Qué hacés
0	Sin mensajes	Seguir
4	Advertencias	Leerlas; en general se sigue
8	Errores: el programa probablemente no hace lo que pensás	Corregir
12 / 16	Errores graves o terminales	Corregir

Cómo encontrar el error:

1. Al final del listado hay un **resumen de diagnósticos** con el número de sentencia de cada mensaje.
2. Cada mensaje tiene la forma `ASMAnnnX`, donde la **letra final** es la gravedad (I, W, E, S...).
3. Los más comunes en un primer ensamblado son **símbolo no definido** y **falta un USING activo** para un operando. Este último es la promesa de

L01 que no se cumplió.

4. **Corregí el primero primero.** Un error al principio suele generar otros diez en cascada.

Los números exactos de cada mensaje los vas a ver en tu listado: los anotamos en la práctica, no de memoria.

3. Correr el monitor: ZTKMONR

```
//MONITOR EXEC PGM=ZTKMONS,PARM='ZTKD* 5 20',TIME=30
//STEPLIB DD DSN=PPCE.ZTK.LOAD,DISP=SHR
//SYSPRINT DD SYSOUT=*,DCB=(RECFM=FB,LRECL=133,BLKSIZE=13300)
```

Detalle	Por qué
PARM= 'ZTKD* 5 20'	Mira los jobs ZTKD*, cada 5 s, durante 20 min (240 vueltas)
TIME=30	Tope de 30 min de CPU: si algo sale mal, no gira para siempre
El job se llama ZTKMONR	No puede empezar con ZTKD: se vería a sí mismo

4. La demo, paso a paso

Paso	Qué hacés	Qué tiene que pasar
1	Subir ZTKMONS.asm y correr ZTKMONA	RC ≤ 4 y PPCE.ZTK.LOAD(ZTKMONS) creado. Nunca se ensambló: los errores aparecen acá
2	Someter ZTKMONR	Queda ACTIVE
3	Someter los jobs de prueba de vivo\demo_desa\ (tabla siguiente)	Corren normal: el monitor sólo mira
4	Al terminar ZTKMONR: python vivo\spool_a_portal.py <JOBID>	Informe en vivo\validaciones\<JOBID>.md y alertas en la pestaña «En vivo»

Job de prueba	Qué hace	Qué tiene que decir el monitor
---------------	----------	--------------------------------

ZTKDOK ×12, SEG entre 20 y 40	BPXBATCH sleep	INI y FIN COMP=NRM CC=0000 ; el portal aprende la duración
ZTKDRC8	IDCAMS borra un DSN que no existe	FIN ... CC=0008 → FALLA
ZTKDABN	EXEC PGM=ZTKNOEXI	FIN ... COMP=ABN CC=S806 → FALLA
ZTKDJCL	DISP=(NUEVO,...) inválido	¿Aparece en POSTEX? (pregunta 3)
ZTKDNUE	sleep 30 , nombre nunca visto	Arranque en frío (umbral 300 s), sin falsa alarma
ZTKDOK SEG=600	El mismo job, durmiendo 10 min	COLGADO al pasar el umbral aprendido
Ráfaga: 50 × ZTKDOK	Muchos seguidos	Ningún evento perdido

Los jobs de prueba no leen ni escriben datos. Antes de someterlos, ajustá la tarjeta JOB (cuenta, CLASS, MSGCLASS) a lo que use DESA.

La demo en modo spool, paso a paso

L04 - ZTKMONS v0 en DESA

Paso	Qué hacés	Qué tiene que pasar
1	Subir ZTKMONS.asm a PPCE.ZTK.ASM(ZTKMONS) y correr ZTKMONA.jc1	ASMA90 con RC ≤ 4 y PPCE.ZTK.LOAD(ZTKMONS) creado. Nunca se ensambó: los errores aparecen acá
2	Someter ZTKMONR.jc1 (PARM 'ZTKD* 5 20' : cada 5 s durante 20 min)	Queda ACTIVE
3	Mientras corre, someter los jobs de prueba: 12 × ZTKDOK (SEG 20–40), ZTKDRC8, ZTKDABN, ZTKDJCL, ZTKDNUE y al final ZTKDOK con SEG=600	Los jobs corren normal: el monitor sólo mira
4	Cuando ZTKMONR termine: python vivo\spool_a_portal.py <JOBID>	Informe en vivo\validaciones\<JOBID>.md y alertas en la pestaña «En vivo»

Job de prueba	Qué simula	Qué debería decir el monitor
ZTKDOK	Job normal de 20–40 s	INI y FIN ... COMP=NRM CC=0000 ; enseña el umbral
ZTKDRC8	Termina con RC 8	FIN ... CC=0008 → alerta FALLA
ZTKDABN	Abend	FIN ... COMP=ABN CC=S...
ZTKDJCL	Error de JCL	¿Aparece en POSTEX? (pregunta 3)
ZTKDNUE	Job nunca visto	Umbral de arranque en frío
ZTKDOK SEG=600	Job que "se cuelga" (10 min)	Alerta COLGADO

Fuente: vivo\host\LEEME.md

5. Qué valida spool_a_portal.py

Chequeo	Contra qué se compara
El SYSPRINT tiene sólo líneas del protocolo, sin +ERR del SSI	El formato de PROTOCOLO.md
El contador de eventos cuadra	+FIN ZTKMON ... EVENTOS=n vs. las +EVT contadas
La hora del monitor es correcta	El IEF403I ... STARTED del JESMSG LG de cada job
El CC de cada FIN es el real	El retcode que da JES para ese jobid
No se perdió ningún job	Todos los ZTKD* sometidos después del monitor
Qué fases reportó JES2	Las líneas +DIAG

Qué valida spool_a_portal.py (✓ / X)

L04 · ZTKMONS v0 en DESA

Chequeo	Contra qué se compara
El SYSPRINT tiene sólo líneas del protocolo, sin +ERR del SSI	El formato de PROTOCOLO.md
El contador de eventos cuadra	+FIN ZTKMON ... EVENTOS=n vs. las líneas +EVT contadas
La hora del monitor es correcta	El IEF403I ... STARTED del JESMSG LG de cada job
El CC de cada +EVT FIN es el real	El retcode que da JES para ese jobid
No se perdió ningún job	Todos los ZTKD* sometidos después del monitor
Qué fases reportó JES2	Las líneas +DIAG (confirma si el ejecutando es ONMAIN)

Analogía: el auditor con el extracto del banco

El monitor es tu **libreta de gastos**; JES es el **extracto oficial**. El script es el auditor que compara línea por línea y marca con ✓ lo que coincide y con X lo que no.

Fuente: vivo\host\LEEME.md (sección "Qué valida el paso 4")

Lámina L04-03_que_valida

6. Las cinco preguntas y dónde se lee cada respuesta

#	Pregunta	Dónde mirar
1	¿ TIME DEC,...,DATETYPE=YYYYMMDD deja la fecha en los bytes 8–11?	La fecha del +HOLA: ¿es la de hoy?
2	¿El que ejecuta tiene STTRPHAZ = 14 ?	Si hay +DIAG FASE y ningún INI, no

3	¿Un error de JCL aparece en POSTEX?	¿Hay +EVT FIN de ZTKDJCL?
4	¿STATVRBO trae STVBXTST / STVBXTET sin APF?	¿Los TI= de los FIN tienen hora?
5	¿Cuánta CPU gasta cada vuelta?	CPU del paso de ZTKMONR (IEF032I) ÷ 240, comparada con P02

Cada respuesta va a 20 - Practicas con su evidencia (JOBID y línea del SYSPRINT). Si alguna sale distinta de lo esperado, se corrige ZTKMONS **y la lámina o la lección que lo suponía.**

7. Preguntas de comprobación

1. ¿Por qué el paso del binder tiene COND=(4,LT,ASM) ? ¿Qué pasaría sin él si el ensamblado diera 8?
2. ¿Por qué ZTKMONR no puede llamarse ZTKDMON?
3. Si el listado tiene 40 errores, ¿por dónde empezás y por qué?
4. ¿Qué te dice que aparezcan +DIAG FASE y ningún +EVT INI ?
5. ¿Cómo calculás la CPU que agrega SSI 80, usando el número de P02?

Mis respuestas

8. Práctica P04: la demo completa

Seguí la sección 4 y traé:

- el RC y los mensajes de ZTKMONA (si hubo errores, el listado);
- el JOBID de ZTKMONR;
- el informe `vivo\validaciones\<JOBID>.md` (lo genera `spool_a_portal.py`; lo corremos juntos);
- tus respuestas a las cinco preguntas de la sección 6.

Relacionado: [L03 - Hablarle a JES por SSI 80](#) · siguiente: [L05 - Diseño del protocolo ZTKMON v1](#).

Fuente: vivo\host\LEEME.md, ZTKMONA.jcl, ZTKMONR.jcl, vivo\demo_desa\LEEME.md y sus JCL.

